

학습 정보 정리

날짜

2026-02-26

작성자

노승우

Alpamayo VRAM Optimization Research

목차 (Table of Contents)

PCIe 양방향 동시 전송 특성

Full-Duplex 이론 vs 실제

실측 데이터

비대칭 원인

CUDA Multi-Stream 동시 실행

레이어 오프로딩 관련 주요 시스템/연구 (2025년 기준)

모델 네이밍 관례

“10B 모델”인데 실제 파라미터가 11.08B인 이유

모델 크기와 메모리

파라미터 수 → 메모리 변환

BF16 vs FP16 차이

양자화와 성능 손실

INT8 / INT4 양자화 시 BF16 대비 성능 영향

CUDA Unified Memory vs iGPU Zero-Copy

공통점

핵심 차이

Unified Memory가 느린 이유

Pinned Memory vs Pageable Memory

이론 0.6초 vs 실제 273초 격차의 원인

CUDA Unified Memory vs device_map="auto" 구분

RTSS 2022 Demand Layering이 iGPU 대상인 이유

HuggingFace 라이브러리

Transformers

Accelerate

관계

Alpamayo 아키텍처의 Expert 모듈

Expert란?

전체 파이프라인에서의 역할

VLM vs Expert 비유

VLM → Expert 정보 전달 방식

Flow Matching이란

디코더 = Expert = Flow Matching 관계

LLM 추론이 Memory-Bandwidth-Bound인 이유

Batch_size=1 Autoregressive 생성 시

FP16 Alpamayo 이론적 최적 추론 시간 (스왑 없음, ≥ 24 GB GPU)

CoT vs CoC

Chain-of-Thought (CoT)

Chain-of-Causation (CoC)

모듈 단위 스와핑 vs device_map="auto" vs 레이어 단위 스와핑

교수님 아이디어: 모듈 단위(stage-level) 동적 스와핑

device_map="auto"와의 차이

모듈 스와핑만으로는 12GB에 불충분한 이유

WSL2 환경 오버헤드

WSL2에서의 GPU 접근 오버헤드

VRAM 점유 구성 (Alpamayo 추론 시)

정적 vs 동적 메모리

KV Cache가 작은 이유

스왑 최적화의 구조적 한계

VRAM 대역폭 vs PCIe 대역폭

파이프라이닝이 효과 없는 이유

성능 한계선

연구 방향 의사결정

배제 대상: 모델 정확도를 떨어뜨리는 방식

추론 시 Activation + Overhead (동적 메모리) 발생 원인

구성 요소

발생 시점

GQA (Grouped Query Attention)

CUDA Unified Memory 동작 원리

물리 메모리 배분 (Alpamayo 22.16GB, VRAM 12GB, RAM 16GB)

Unified Memory vs device_map="auto" 비교

학습 정보 정리

작업 중 질의응답을 통해 습득한 지식과 정보를 정리하는 문서입니다. 마지막 업데이트:
2026-02-26

PCIe 양방향 동시 전송 특성

Full-Duplex 이론 vs 실제

- PCIe는 물리적으로 full-duplex (TX/RX 별도 배선)
- PCIe Gen3 x16 이론: 각 방향 ~15.75 GB/s, 양방향 합계 ~31.5 GB/s
- 실제 양방향 동시 전송 시 상당한 성능 저하 발생

실측 데이터

환경	단방향 H2D	단방향 D2H	양방향 H2D	양방향 D2H	H2D 감소 율
PCIe 3.0 RTX 3080	~12 GB/s	~12 GB/s	~9.45 GB/s	~9.45 GB/s	-21%
PCIe 4.0 A100 (사례 1)	~24.8 GB/s	~25.9 GB/s	~11 GB/s	~24.2 GB/s	-56%

비대칭 원인

- PCIe root complex arbitration이 읽기(D2H)를 쓰기(H2D)보다 우선
- CPU 메모리 컨트롤러 내부 버스 경합
- CUDA 드라이버 스케줄링 정책, IOMMU 오버헤드

CUDA Multi-Stream 동시 실행

- Compute Capability ≥ 2.0 + `asyncEngineCount` ≥ 1 이면 compute + H2D 오버랩 가능

- `asyncEngineCount > 1`이면 H2D + D2H + compute 3-way 오버랩도 가능
- 반드시 **pinned memory 사용 필요** (pageable → async 불가)
- 현대 GPU (Kepler 이후)는 2개 이상 DMA copy engine 보유

참고: 파라미터 오프로딩 추론에서는 D2H가 불필요하다. CPU에 파라미터 원본이 유지되므로, 사용 완료된 레이어는 GPU free만 하면 됨. 따라서 2-stream(compute + H2D)만으로 충분하다.

레이어 오프로딩 관련 주요 시스템/연구 (2025년 기준)

시스템	비동기 H2D	D2H 사용	특징
DeepSpeed ZeRO-Inference	O	불필요 (파라미터 추론)	다중 GPU PCIe 집합 대역폭
FlexGen	O	O (KV Cache/Activation)	LP 기반 최적 텐서 배치
HF Accelerate	최근 도입	GPU free 사용	Hook 기반, 범용 프레임워크
PIPO (2025)	O	O (KV Cache saving)	세밀한 4종 태스크 파이프라인
NEO (MLSys 2025)	O	O (KV Cache)	KV cache + attention 오프로딩
vLLM	O	O (KV Cache)	PagedAttention, 서빙 최적화
llama.cpp	X (정적 배치)	X	mmap, MoE 특화 최적화

참고: 파라미터 오프로딩 추론에서 D2H는 불필요 (CPU에 원본 보유). D2H가 필요한 경우: KV Cache 보존, Activation 저장(학습), 체크포인트 등.

모델 네이밍 관례

“10B 모델”인데 실제 파라미터가 11.08B인 이유

- 모델명의 숫자는 정확한 파라미터 수가 아닌 **사이즈 클래스(규모 범주)** 표시
- 업계 관례: LLaMA-7B(실제 6.74B), LLaMA-70B(실제 68.98B), Qwen-7B(실제 7.62B) 등
- 논문/문서 작성 시: 모델명은 공식 명칭(Alpamayo-R1-**10B**) 그대로, 파라미터 수 언급 시에는 정확한 수치(**11.08B**) 사용

모델 크기와 메모리

파라미터 수 → 메모리 변환

- 파라미터 수(B) = 모델의 가중치 숫자 개수 (예: 11.08B = 110.8억 개)
- 메모리 = 파라미터 수 × 데이터 타입 바이트 크기

데이터 타입	바이트/파라미터	11.08B 모델 기준
FP32	4 bytes	~44.3 GB
BF16 / FP16	2 bytes	~22.2 GB
INT8	1 byte	~11.1 GB
INT4	0.5 bytes	~5.5 GB

BF16 vs FP16 차이

둘 다 16비트(2바이트)지만 비트 배분이 다름:

	FP16 (float16)	BF16 (bfloat16)
부호	1비트	1비트
지수(exponent)	5비트	8비트
가수(mantissa)	10비트	7비트
표현 범위	$\pm 65,504$	$\pm 3.4 \times 10^{38}$ (FP32와 동일)
정밀도	더 정밀	덜 정밀

- **FP16**: 정밀도 높지만 범위 좁음 → 큰 값에서 오버플로우 위험 (loss scaling 필요)
- **BF16**: 정밀도 낮지만 범위가 FP32와 동일 → 딥러닝에서 안정적
- BF16은 Google이 TPU용으로 설계, NVIDIA Ampere(RTX 30xx) 이후 GPU 지원
- **딥러닝에서 BF16 선호 이유**: 가중치/gradient의 정밀도는 소수점 10자리까지 불필요하지만, 값의 범위는 넓어야 함. FP16은 $\pm 65,504$ 초과 시 오버플로우(inf) 발생 → loss scaling 필요. BF16은 FP32와 동일 범위라 loss scaling 없이 안정적 동작. 정밀도 손실(10→7비트)은 모델 품질에 거의 영향 없음이 실험적으로 확인됨

양자화와 성능 손실

INT8 / INT4 양자화 시 BF16 대비 성능 영향

양자화	메모리 절감	정확도 손실 (일반 LLM)	비고
BF16 (기준)	0%	0%	기준선
INT8	50%	~0.1-1%	거의 무손실
INT4	75%	~1-5%	대부분 허용 범위
INT4 (나쁜 경우)	75%	~5-10%	소형 모델이나 특수 태스크

- 모델이 클수록(10B+) 양자화 손실에 강함, 소형(1-3B)은 손실 큼
- 추론 속도는 오히려 빨라짐 (메모리 대역폭 병목 해소, INT4가 BF16 대비 2-3x 빠름)
- 양자화 방법에 따라 차이: RTN(단순, 손실 큼) < GPTQ/AWQ(캘리브레이션, 손실 적음)
- 도메인 특화 태스크(자율주행 궤적 예측 등)는 일반 벤치마크와 품질 기준이 다를 수 있음

CUDA Unified Memory vs iGPU Zero-Copy

공통점

- CPU와 GPU가 같은 메모리 주소 사용, 명시적 복사 없이 데이터 접근

핵심 차이

	iGPU Zero-Copy	CUDA Unified Memory (dGPU)
물리 메모리	CPU/GPU 동일한 물리 RAM 공유	CPU RAM과 GPU VRAM 물리적 분리
데이터 이동	실제 복사 없음 (진짜 zero-copy)	페이지 폴트 시 PCIe로 마이그레이션
VRAM 초과 시	해당 없음 (RAM=VRAM)	자동 스와핑 (페이지 폴트 기반)
대표 HW	Jetson, Apple M시리즈, Intel iGPU	RTX/Quadro/A100 등 discrete GPU

Unified Memory가 느린 이유

1. 페이지 폴트 기반 — GPU 실행 중단 → 드라이버 트랩 → PCIe 전송 → 재개
2. 양방향 eviction — VRAM 포화 시 기존 페이지 내보내야 새 페이지 가져옴
3. 소규모 페이지 단위 전송 (4KB~2MB) → PCIe 대역폭 활용률 저하
4. 접근 패턴 예측 불가 → 프리페칭 불가

Pinned Memory vs Pageable Memory

- **Pageable**: OS가 디스크로 스왑 가능 → GPU DMA 전 임시 pinned 버퍼로 복사 필요 (추가 단계)
- **Pinned**: OS가 스왑 불가하도록 고정 → GPU DMA가 직접 접근, 비동기 전송 가능
- Pinned의 장점: 중간 복사 제거 + CPU 개입 없이 DMA 독립 전송 (GPU 연산과 동시 실행)
- Pinned의 단점: 시스템 RAM 고정 점유 → 과다 사용 시 OS 메모리 부족 위험
- WSL2에서 D2H: pageable 2.48 GB/s vs pinned 11.8 GB/s (4.8배 차이, 가상화 오버헤드)
- **현실 제약**: 시스템 RAM(15GB) < 모델(22GB)이므로 전체를 pinned로 올릴 수 없음 → 소규모 pinned 버퍼(1~2GB)를 할당하고 레이어를 번갈아 올리는 방식이 현실적 (On-Demand Layering + 스왑 최적화 결합)

이론 0.6초 vs 실제 273초 격차의 원인

- **이론**: 9.5GB를 한 덩어리로 연속 전송 가정 ($9.5\text{GB} \div 16\text{GB/s} \approx 0.6\text{초}$)
- **실제**: 토큰 생성마다 레이어 접근 → 페이지 폴트 → 소규모 전송 반복
- 반복 횟수: $256\text{토큰} \times 36\text{레이어} = 9,216\text{번} + \text{레이어 접근}$
- per-transfer 오버헤드: 0.36ms/건 (WSL2 측정)
- 양방향 경합: evict(내보내기) + fetch(가져오기) 동시 발생
- 연구 방향 1~3의 핵심: 페이지 폴트 기반 자동 스와핑 → 명시적/계획적 전송으로 대체

CUDA Unified Memory vs device_map="auto" 구분

- **CUDA Unified Memory**: GPU 드라이버 레벨의 자동 페이지 스와핑. `.to("cuda")` 로 VRAM 초과 할당 시 투명하게 동작. 베이스라인에서 12GB로 Alpamayo가 돌아간 이유.
- **device_map="auto"**: Accelerate 라이브러리가 레이어를 GPU/CPU에 미리 나눠 배치. Alpamayo에서는 커스텀 토큰 처리 비호환으로 추론 실패.
- 두 개는 완전 다른 메커니즘. 현재 동작하는 것은 Unified Memory이고, 그것이 느리기 때문에 최적화 연구 중.
- **device_map="auto"가 Alpamayo에서 실패하는 이유**: Alpamayo는 `fuse_traj_tokens()` 라는 커스텀 연산에서 VLM 출력 텐서와 trajectory 토큰 테이블을 `masked_scatter` 로 합침. device_map이 텐서를 GPU/CPU에 분산 배치하면 이 연산에서 디바이스 불일치 에러 발생. 해결하려면 소스코드 수정이 필요 → 커스텀 오프로딩 구현이 연구 핵심 과제.

RTSS 2022 Demand Layering이 iGPU 대상인 이유

- iGPU는 CPU RAM을 공유하므로 “CPU로 오프로딩” 불가
- 대신 NVMe SSD로 오프로딩하는 새로운 접근이 필요

HuggingFace 라이브러리

Transformers

- HuggingFace의 모델 로딩/추론 프레임워크
- `from_pretrained()` 로 모델 다운로드, config 파싱, 가중치 로딩 등 처리
- LLM/VLM 생태계의 사실상 표준

Accelerate

- HuggingFace의 분산/멀티 디바이스 실행 지원 라이브러리

- `device_map="auto"` 등 GPU/CPU 자동 배치 기능 제공
- 모델이 단일 GPU에 안 들어갈 때 여러 디바이스에 분배
- 배치 순서: GPU VRAM부터 채움 → 초과분은 CPU RAM → 그래도 부족하면 디스크
- 추론 시 CPU에 있는 레이어를 실행 차례에 GPU로 이동 → 완료 후 CPU로 반환 (자동)

관계

사용자 코드 → Transformers (모델 로딩/추론) → Accelerate (디바이스 배치)
→ PyTorch (GPU 연산)

Alpamayo 아키텍처의 Expert 모듈

Expert란?

- Diffusion Decoder의 핵심 엔진 — **denoising 트랜스포머**
- VLM의 텍스트 설정(text_config)을 복제해서 만든 별도의 트랜스포머 모델
- VLM과 구조는 비슷하지만 독립된 가중치를 가짐

전체 파이프라인에서의 역할

1. **Vision Encoder** (1.15GB): 이미지 → 비전 토큰
2. **VLM** (15.17GB): 비전 토큰 + 텍스트 → Chain-of-Thought 추론 (텍스트 생성) + 프롬프트 캐시 생성
3. **Expert** (Diffusion 포함 4.56GB): noisy action → 프롬프트 캐시 참고하며 반복 denoising → 최종 궤적 출력

VLM vs Expert 비유

- **VLM**: “이 상황에서 어떻게 해야 할지 생각한다” (추론/텍스트)
- **Expert**: “그 생각을 바탕으로 실제 행동(궤적)을 만든다” (action 생성)

VLM → Expert 정보 전달 방식

- VLM이 넘기는 것은 hidden state가 아니라 **KV Cache (프롬프트 캐시)**
- Hidden state 전달: 마지막 레이어 출력을 1회 전달 (정보 제한적)
- KV Cache 전달: VLM이 생성한 **모든 토큰의 Key/Value**를 Expert가 attention으로 참조 (매 스텝마다 전체 맥락 활용)
- Expert는 denoising 할 때마다 VLM의 전체 추론 과정을 들여다볼 수 있음

Flow Matching이란

- 노이즈 → 깨끗한 데이터로 가는 “**경로(flow)**”를 **벡터 필드로 학습**하는 생성 모델
- 시간 $t=0$ (순수 노이즈)에서 $t=1$ (깨끗한 궤적)까지, 매 스텝마다 Expert가 이동 방향(벡터 v)을 예측
- 핵심 수식: $x_{next} = x + dt \times v$ (Euler 적분)
- **기존 Diffusion과의 차이**: Diffusion은 노이즈(ϵ)를 예측하고 SDE 기반(확률적, 스텝 많이 필요), Flow Matching은 속도(v)를 예측하고 ODE 기반(결정적, **적은 스텝으로 가능**)
- Alpamayo에서 사용하는 이유: 자율주행의 실시간 요구 → 10스텝 정도로 고품질 궤적 생성 가능
- 디퓨전 스텝 수 = Expert 호출 횟수 → 스텝 축소가 곧 추론 시간 단축

디코더 = Expert = Flow Matching 관계

- 셋은 별개가 아니라 하나의 디코딩 파이프라인
- **Flow Matching**: 디코딩 방법론 (어떻게 생성할 것인가, Diffusion의 ODE 변형)
- **Expert**: 디코딩 엔진 (각 스텝에서 벡터 필드를 예측하는 트랜스포머)
- **Action Decoder**: 이 둘을 합친 기능적 명칭
- Expert는 Diffusion 샘플러의 `step_fn` 으로 호출됨
- 각 denoising 스텝마다 Expert 트랜스포머가 실행
- 독립적으로 동작하지 않고 Diffusion 프로세스의 일부

LLM 추론이 Memory-Bandwidth-Bound인 이유

Batch_size=1 Autoregressive 생성 시

- 매 토큰마다 전체 모델 가중치를 VRAM에서 한 번 읽어야 함
- RTX 3080 Ti 기준 (FP16 34.1 TFLOPS, 메모리 912 GB/s):
- 가중치 읽기: $16.45 \text{ GB} / 912 \text{ GB/s} \approx 18.0\text{ms}/\text{토큰}$ (병목)
- 연산: $7.58\text{B} \times 2 \text{ FLOP} / 34.1 \text{ TFLOPS} \approx 0.44\text{ms}/\text{토큰}$ (읽기의 2.4%)
- 연산 시간은 가중치 읽기 시간에 완전히 은닉됨
- 따라서 모델 크기를 줄이면 속도도 비례하여 빨라짐 (데이터 읽기량 감소)

FP16 Alpamayo 이론적 최적 추론 시간 (스왑 없음, $\geq 24\text{GB GPU}$)

- VLM: $16.45 \text{ GB}/\text{토큰} \times 256\text{토큰} / 912 \text{ GB/s} \approx 4.62\text{s}$

- Expert: $4.56 \text{ GB} \times 10\text{스텝} / 912 \text{ GB/s} \approx 0.05\text{s}$
- 기타 (Vision, KV Cache): $\sim 0.3\text{s}$
- **합계: $\sim 5.0\text{s}$** (현재 273.79s의 98%가 스왑 오버헤드)

CoT vs CoC

Chain-of-Thought (CoT)

- 일반 LLM 기법 (Wei et al., 2022)
- “생각의 연쇄” — 논리적 추론을 단계별로 전개

Chain-of-Causation (CoC)

- Alpamayo 논문에서 제안한 자율주행 특화 개념
- “인과의 연쇄” — 원인→결과→행동의 인과관계를 명시적으로 추론
- CoT의 자율주행 특화 버전. “무엇을 해야 하는지”가 아니라 “**왜** 그래야 하는지”가 핵심
- 코드에서는 `<|cot_start|>`, `<|cot_end|>` 토큰을 사용하여 구현 레벨에서는 혼용됨

모듈 단위 스와핑 vs device_map="auto" vs 레이어 단위 스와핑

교수님 아이디어: 모듈 단위(stage-level) 동적 스와핑

- 비전 인코더(1.15GB) 로드 → 실행 → 언로드 → VLM(15.17GB) 로드 → 실행 → 언로드 → Diffusion(4.56GB) 로드 → 실행 → 언로드
- Peak VRAM = $\max(1.15, 15.17, 4.56) = \mathbf{15.17GB}$ (전체 21GB 대비 감소)
- 한 번에 하나의 모듈만 VRAM에 존재

device_map="auto"와의 차이

	교수님 아이디어 (모듈 스와핑)	device_map="auto"	연구 방향 1&2 (레이어 + 모듈)
스와핑 단위	모듈(stage)	레이어	레이어 + 모듈
배치 시점	동적 (사용 시 로드/언로드)	정적 배치 + 동적 이동	동적 + 파이프라이닝
Peak VRAM	~15.17GB	설정에 따라 다름	12GB 목표
Alpamayo 호환	커스텀 구현 필요	X (fuse_traj_tokens 실패)	커스텀 구현 필요

모듈 스와핑만으로는 12GB에 불충분한 이유

- VLM 단독이 15.17GB → 12GB VRAM 초과
- 결국 VLM 내부에서도 **레이어 단위** 스와핑이 추가로 필요
- 이것이 연구 방향 1(On-Demand Layering)과 방향 2(Memory Pipelining)에서 탐색한 내용
- 최종적으로 **모듈 단위 + 레이어 단위** 복합 스와핑이 해법

WSL2 환경 오버헤드

WSL2에서의 GPU 접근 오버헤드

현재 실험 환경은 네이티브 Linux가 아닌 WSL2. GPU 접근 시 추가 오버헤드 발생.

항목	WSL2 (현재)	네이티브 Linux (예상)	차이
Pageable D2H 대역폭	2.48 GB/s	~11 GB/s	4.5배 느림
Pinned D2H 대역폭	11.8 GB/s	~12+ GB/s	유사
per-transfer 고정 오버헤드	0.36 ms	~0.05 ms	7배
cudaMemPrefetchAsync(CPU)	미지원	지원	—

- Pageable D2H 비정상 감속: WSL2 가상화 레이어의 영향
- Pinned memory는 WSL2에서도 정상 → pinned 기반 구현이 핵심

- 네이티브 Linux 전환 시 **약 40% 성능 개선** 예상
- cudaMemPrefetchAsync(CPU) 미지원으로 CUDA Managed Memory 프리페치 경로 차단

VRAM 점유 구성 (Alpamayo 추론 시)

정적 vs 동적 메모리

- **정적 (93%)**: 모델 파라미터 = 22.16 GB
- VLM Language Model: 15.17 GB (63.7%)
- Expert: 4.56 GB (19.1%)
- VLM LM Head: 1.28 GB (5.4%)
- Vision Encoder: 1.15 GB (4.8%)
- **동적 (7%)**: 추론 중 변하는 메모리 = ~1.66 GB
- KV Cache: ~0.56 GB (2.4%) — GQA 덕에 Full MHA의 1/4
- Activation + CUDA overhead: ~1.1 GB (4.6%) — SDPA로 $O(n^2)$ 회피

KV Cache가 작은 이유

- Qwen3-VL은 GQA(Grouped Query Attention) 사용: KV Head 8개 (Attention Head 32개 대비 1/4)
- 4,470 토큰 기준 KV Cache: ~559 MB (Full MHA였다면 ~2.2 GB)
- 토큰당 KV Cache: ~144 KB (36 layers $\times$ 2 $\times$ 8 heads $\times$ 128 dim $\times$ 2 bytes)

스왑 최적화의 구조적 한계

VRAM 대역폭 vs PCIe 대역폭

- VRAM 내부 대역폭: 912 GB/s
- PCIe Gen3 (pinned): 8.5 GB/s
- 격차: 107배

파이프라이닝이 효과 없는 이유

- 레이어 실행 시간: ~0.5ms
- 레이어 전송 시간: ~45ms (PCIe)

- 전송이 실행보다 90배 느림 → 전송을 연산으로 은닉 불가능

성능 한계선

전략	추론 시간	이론 최적 대비
이론 최적 (스왑 없음)	~5s	1x
최선의 스왑 최적화	~80-150s	16-30x
현재 Baseline	273.79s	55x

→ 12GB VRAM + PCIe Gen3에서 스왑 없는 성능에 근접하는 것은 구조적으로 불가능

연구 방향 의사결정

배제 대상: 모델 정확도를 떨어뜨리는 방식

- 양자화 (INT8, INT4, 하이브리드 양자화 등)
- 레이어 가지치기 (layer pruning)
- 동적 해상도 축소
- 모델 구조 변경 전반

→ 연구 초점: 모델 자체는 그대로 두고, 시스템/메모리 레벨에서 최적화 (On-Demand Layering, 메모리 파이프라이닝, CPU-GPU 스왑 최적화 등)

추론 시 Activation + Overhead (동적 메모리) 발생 원인

Alpamayo 추론 시 정적 파라미터(22.16GB) 외에 ~1.10GB의 동적 메모리가 추가 점유된다.

구성 요소

1. **Activations (~0.3-0.5 GB):** Forward pass 중 현재 레이어의 중간 텐서 - Hidden state
`[batch, seq_len, 4096]` (레이어 간 전달) - Attention score 행렬 ($Q \cdot K^T$), softmax 결과 - MLP gate/up projection 중간값 ($4096 \rightarrow 11008 \rightarrow 4096$) - 추론이므로 backprop용 전체 보관 불필요, 현재 1개 레이어 분만 필요

2. **CUDA Context / Runtime (~0.3-0.5 GB)**: GPU 드라이버 + CUDA 런타임 고정 메모리 - cuDNN 커널 워크스페이스 (최적화된 attention/conv 커널용) - `torch.cuda.init()` 시점에 할당
3. **PyTorch Caching Allocator Fragmentation (~0.1-0.3 GB)** - `cudaFree` 호출 대신 메모리 풀 캐싱 → 단편화로 실사용보다 점유 큼

발생 시점

- CUDA Context: 모델 로드 시 고정 할당
- Activations: 매 레이어 forward마다 할당/해제 반복, 레이어 순차 실행이라 동시 1개분만 필요
- Fragmentation: 할당/해제 누적으로 점진 증가, `empty_cache()` 로 일부 회수 가능

GQA (Grouped Query Attention)

MHA(Multi-Head Attention)의 메모리 효율적 변형. KV Cache 크기를 대폭 줄인다.

- **MHA**: Q, K, V 모두 동일한 head 수 (예: 32 heads → K 32개, V 32개)
- **GQA**: Q head 수는 유지, K/V는 소수 그룹으로 공유 (예: Q 32 heads / KV 8 groups → KV 1/4)
- Alpamayo VLM (Qwen3-VL-8B): Q 32 heads / KV 8 groups → KV Cache 0.56GB (전체 2.4%)
- 일반 LLM (Full MHA + 긴 컨텍스트): KV Cache 수~수십 GB (30-50%+)

CUDA Unified Memory 동작 원리

CUDA Unified Memory는 GPU VRAM + CPU RAM을 단일 가상 주소 공간으로 통합한다.

물리 메모리 배분 (Alpamayo 22.16GB, VRAM 12GB, RAM 16GB)

- `.to("cuda")` 시 가상 주소 22.16GB 할당, 물리적으로는 VRAM(12GB) + CPU RAM(~10GB)에 분산
- GPU가 페이지 접근 시 VRAM에 없으면 **page fault** → 드라이버가 CPU RAM에서 VRAM으로 이동 (다른 페이지 evict)
- CPU RAM마저 부족하면 OS 디스크 swap까지 사용 (극단적으로 느림)

- 페이지 단위: 64KB, 수천 회 page fault로 273.79s 소요

Unified Memory vs device_map="auto" 비교

항목	Unified Memory	device_map="auto"
관리 주체	CUDA 드라이버 (HW/OS 레벨)	HuggingFace Accelerate (프레임워크)
배치 단위	페이지 (64KB)	레이어 (수백 MB)
배치 시점	런타임 on-demand (page fault)	로드 타임 정적 결정
이동 방식	반응적 (접근 시 이동)	능동적 (forward hook으로 이동)
정확성	항상 동작 (느릴 뿐)	Alpamayo에서 실패 (cross-device 연산 오류)
제어 가능성	불가 (드라이버 자동)	가능 (레이어별 배치 지정)

핵심: UM은 “전부 GPU에 있는 척” 페이지 단위 투명 스왑, device_map은 “레이어별 명시 배치”. UM은 항상 동작하지만 제어 불가, device_map은 제어 가능하지만 cross-device 연산에서 실패.